Phase One architecture and deployment runbook: services, scaling topology, and zero-downtime rollout sequence

Computer
Shared student-faculty database schema: entity relationship diagram and FERPA role-based access control policies
Computer
Show the unified navigation component system
Outline the legacy record migration strategy
Detail the access control and FERPA policy layer

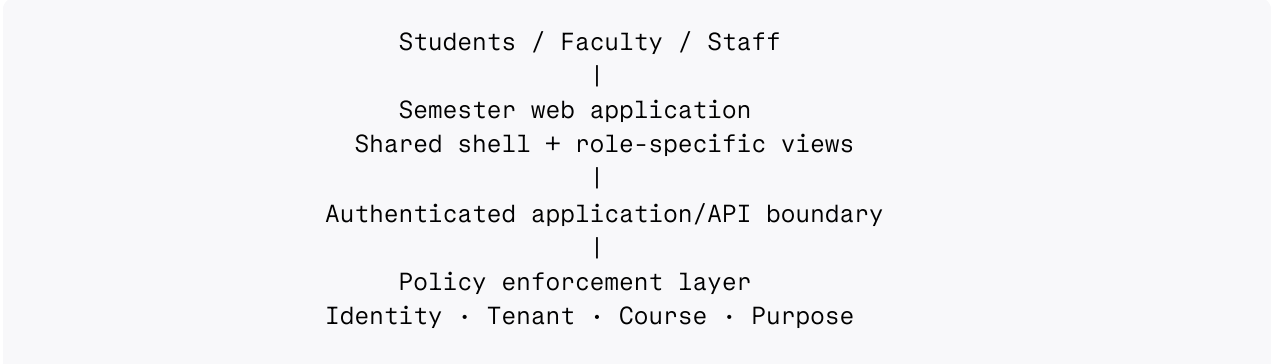
Phase One should deploy Semester as a shared academic platform with role-specific student and faculty workspaces, a server-enforced policy layer, canonical academic records, and durable background processing. The architecture should support **continuous** rollout through backward-compatible database changes—not rely on an application rollback to undo migrations.

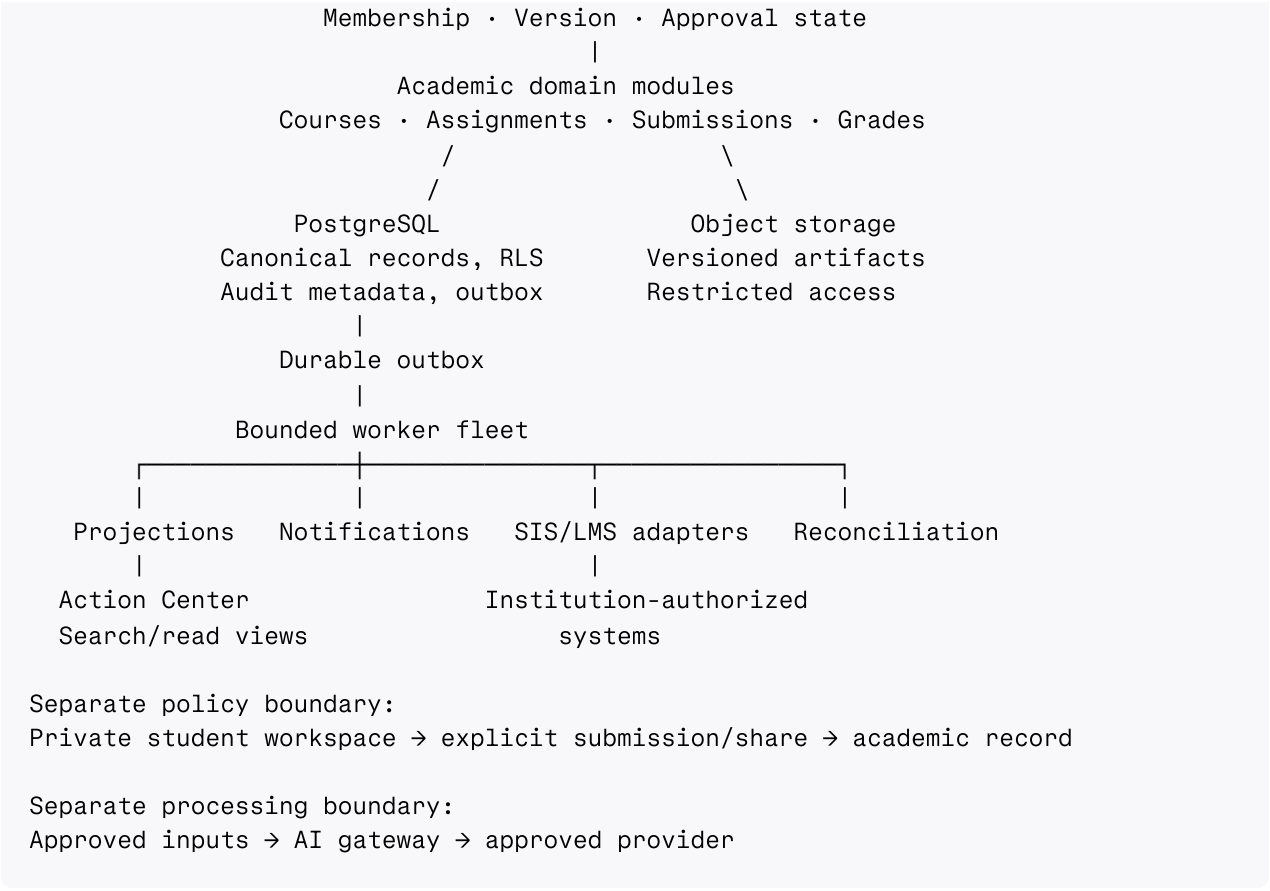
Below is the proposed implementation specification. It expands the previous plan with scaling topology, rollout mechanics, an entity relationship diagram, access policies, navigation structure, and migration controls. It is not a claim that these services or policies are already implemented or deployed. The project's release catalog explicitly requires implementation, isolation, testing, accessibility, monitoring, support, and release evidence before calling a capability complete.[1]

1. Phase One architecture: services and scaling topology

1.1 Service topology

Start with a modular application and independently operated workers. Avoid creating a microservice for every product module.





1.2 Services and responsibilities

Logical service	Responsibility	Scaling unit	Failure behavior
Web application	Navigation, accessible interfaces, private-work experience	Hosted application instances/requests	Preserve drafts; explain unavailable server actions
Application/API layer	Validate requests and execute domain commands	Stateless request handling	Reject unauthorized or incompatible commands
Policy layer	Evaluate identity, scope, purpose, authority and obligations	Request evaluation; database enforcement	Deny protected operations when essential evidence is unavailable
PostgreSQL	Canonical academic records, constraints, transactions, outbox	Database capacity and query efficiency	Do not fabricate completion when commit outcome is unknown
Object storage	Submission artifacts and approved shared materials	Storage and request throughput	Keep upload, artifact verification and submission acceptance distinct
Worker fleet	Event delivery, projections, notifications and integration jobs	Worker concurrency and queue partitions	Retry safely; quarantine exhausted or ambiguous jobs
Integration adapters	Exchange authorized records with institutional systems	Per-provider concurrency and rate limits	Show pending/stale; reconcile uncertain outcomes

Logical service	Responsibility	Scaling unit	Failure behavior
Reconciliation service	Compare Semester with authoritative sources	Scheduled/bounded jobs	Route consequential discrepancies to authorized humans
Observability	Metrics, traces, alerts and operational diagnosis	Telemetry throughput	Avoid sensitive record contents in ordinary logs
AI gateway	Permit only approved input and tool use	Bounded requests and provider quotas	Fail without exposing records or bypassing policy

These are proposed service boundaries. Some can run within the same deployable application; logical separation does not require separate hosting.

1.3 Scaling rules

Web and API

- Keep request handlers stateless.
- Keep user/session identity in authenticated tokens and authoritative server records.
- Bound request concurrency against database capacity.
- Apply per-user, per-tenant and per-operation limits.
- Avoid expensive cross-domain joins on every page load.
- Keep cached academic responses scoped to the actor and institutional context.

For direct PostgreSQL access from serverless functions, Supabase documents transaction-mode pooling for short-lived connections. Pooling configuration must be selected for the actual runtime and client rather than copied blindly. [2]

Database

Scale in this order:

1. Correct indexes and query plans.
2. Bounded pagination and payloads.
3. Connection pooling and request concurrency limits.
4. Efficient read models for queues and dashboards.
5. Additional database capacity.
6. Read replicas only where supported and where stale reads are acceptable.

Do not evaluate a consequential permission, revocation, grade release, or enrollment action against a lagging replica.

Workers

- Use durable leases and idempotency.
- Partition concurrency by tenant and external provider.
- Cap worker connections separately from web/API connections.
- Scale from queue age and completion latency—not queue length alone.
- Apply fair scheduling so one institution cannot monopolize the fleet.
- Preserve a separate reconciliation queue for uncertain external outcomes.
- Stop retrying errors that require a person or changed authorization.

Capacity acceptance

Before launch, approve a measured capacity envelope:

```
Concurrent active sessions
Peak protected requests per second
Database connection budget
Largest supported class/cohort
Artifact upload sizes
Outbox processing latency
Projection freshness
External-provider quotas
Recovery and reconciliation duration
```

No numerical capacity claim is established here. Values must come from target-specific tests.

2. Zero-downtime rollout runbook

“Zero downtime” is a rollout objective, not a guarantee. It depends on schema compatibility, migration lock behavior, provider availability, client version handling, and rehearsed recovery.

2.1 Compatibility contract

During rollout, both application versions must work:

```
Application N     ─┐
                   ├── Expanded database schema
Application N+1   ─┘
```

Required conditions:

- N+1 can read existing records.
- N can tolerate additive changes.
- New mandatory fields have a safe transition strategy.
- Events remain understandable to active consumers.
- Old browser sessions cannot corrupt new records.

- Rollback to N remains valid until the compatibility window closes.

Vercel Rolling Releases can distribute page loads between the old and new deployment. Skew Protection can keep supported framework-managed requests aligned with the deployment that served the page. It does not eliminate shared-database or external-API compatibility requirements. Support must be verified for Semester's actual framework and project configuration. [3] [4]

2.2 Expand → backfill → switch → contract

Step	Action	Required verification	Recovery
0	Freeze release manifest	Candidate, schema, workers, configuration and tenant scope identified	Do not start
1	Verify backups and rollback compatibility	Restore evidence and previous application compatibility	Hold release
2	Expand schema	Additive tables/columns; reviewed indexes and lock behavior	Leave harmless additive structures
3	Deploy compatible consumers	Workers accept current and new event versions	Revert compatible worker version
4	Deploy N+1 with features disabled	Protected workflows pass on hosted target	Return application traffic to N
5	Run resumable backfill	Checkpoints, counts, relationships and exceptions reconcile	Pause; resume or repair
6	Shadow-read new representation	New and old outputs compared without changing user decisions	Keep old read path
7	Canary application traffic	Technical and workflow metrics pass	Abort canary
8	Activate approved tenant capabilities	Target-specific authority and data checks pass	Disable capability; preserve records
9	Expand rollout	Agreed observation windows pass	Return to compatible stable release
10	Close compatibility window	Old sessions/consumers addressed; rollback decision accepted	Delay cleanup
11	Contract schema in a later release	Removed fields no longer used; recovery plan reviewed	Forward repair or approved restoration

Do not combine destructive schema cleanup with the first production deployment of the new application.

2.3 Illustrative canary stages

A possible sequence is internal validation → 5% → 25% → 100%. These percentages are proposed, not an existing Semester configuration.

At each stage, assess:

- Authentication and authorization failures.

- Submission acceptance failures.
- Grade-release failures.
- Database latency and connection saturation.
- Oldest outbox item and projection lag.
- Artifact upload failures.
- External integration uncertainty.
- Support incidents.
- Cross-tenant or unauthorized-disclosure findings.

Any confirmed tenant-isolation failure stops the affected rollout immediately. Other thresholds must be agreed before deployment.

2.4 Rollback rules

Application rollback must not:

- Delete valid submissions.
- Reverse a released grade without domain authority.
- Repeat an external registration or payment.
- Erase audit or reconciliation evidence.
- Restore revoked access.

For uncertain external execution, disable repeat actions and reconcile the authoritative result.

3. Shared student–faculty schema and ER diagram

3.1 Entity relationship diagram

The following is a proposed schema model.

```
erDiagram
    INSTITUTION ||--o{ MEMBERSHIP : contains
    ACCOUNT ||--o{ MEMBERSHIP : holds
    MEMBERSHIP ||--o{ ROLE_GRANT : receives

    INSTITUTION ||--o{ TERM : defines
    INSTITUTION ||--o{ COURSE : owns
    COURSE ||--o{ OFFERING : has
    TERM ||--o{ OFFERING : schedules

    OFFERING ||--o{ TEACHING_ASSIGNMENT : authorizes
    MEMBERSHIP ||--o{ TEACHING_ASSIGNMENT : serves

    OFFERING ||--o{ ENROLLMENT : contains
    MEMBERSHIP ||--o{ ENROLLMENT : participates

    OFFERING ||--o{ ASSIGNMENT : contains
```

```
    ASSIGNMENT ||--o{ ASSIGNMENT_VERSION : versions

    ASSIGNMENT ||--o{ SUBMISSION : receives
    ENROLLMENT ||--o{ SUBMISSION : owns
    SUBMISSION ||--o{ SUBMISSION_VERSION : versions
    ASSIGNMENT_VERSION ||--o{ SUBMISSION_VERSION : references

    SUBMISSION_VERSION ||--o{ GRADE_ENTRY : evaluated
    TEACHING_ASSIGNMENT ||--o{ GRADE_ENTRY : grades
    GRADE_ENTRY ||--o| GRADE_RELEASE : releases

    ACCOUNT ||--o{ PRIVATE_DOCUMENT : owns
    INSTITUTION ||--o{ SOURCE_BINDING : maps
    INSTITUTION ||--o{ AUDIT_EVENT : scopes
    INSTITUTION ||--o{ OUTBOX_EVENT : scopes
```

The private-document relationship deliberately stops at the account. An optional course association must not grant faculty access.

3.2 Core tables

Table	Key fields	Important constraint
`memberships`	Institution, account, status, validity	Current membership alone is insufficient for course access
`role_grants`	Membership, capability bundle, scope, expiry, grantor	Scope and expiry enforced server-side
`offerings`	Institution, course, term, section, status	Institution-scoped identity
`teaching_assignments`	Offering, membership, teaching role, validity	Grading authority separate from viewing authority
`enrollments`	Offering, membership, status, authoritative source	Student cannot self-write authoritative enrollment
`assignments`	Offering, lifecycle, current version, record version	Publication through validated command
`assignment_versions`	Assignment, instructions, due date, rubric, AI policy	Published historical versions preserved
`submissions`	Assignment, enrollment, status, current version	Enrollment and assignment must share offering
`submission_versions`	Artifact, checksum, assignment version, received time	Accepted version immutable
`grade_entries`	Submission version, grader scope, score, feedback, status	Draft access limited to permitted graders
`grade_releases`	Grade entry, releaser, time, receipt	Student sees released snapshot
`private_documents`	Owner, content reference, version	No automatic institutional visibility
`source_bindings`	Source key, canonical ID, source version, observation time	Unique within source/institution/object type

Table	Key fields	Important constraint
`audit_events`	Actor, action, target, outcome, policy reference	Protected, append-oriented evidence
`outbox_events`	Aggregate, version, event type, delivery status	Transactionally paired with domain change

3.3 Database integrity

For institution-owned parent records:

```
UNIQUE (institution_id, id)
```

For child references:

```
FOREIGN KEY (institution_id, offering_id)
  REFERENCES offerings (institution_id, id)
```

Use equivalent composite relationships throughout the academic model. This is necessary because a simple object foreign key does not, by itself, prove tenant alignment.

Additional invariants:

- Submission enrollment belongs to the assignment's offering.
- Submission version references the assignment version presented at submission.
- Grader authority is valid for the relevant offering.
- Current-version pointers refer to versions of the same parent.
- Source imports cannot overwrite native records outside their approved authority.
- Released feedback is a stable version, not a mutable live view of a grading draft.

4. Access control and FERPA policy layer

4.1 FERPA is not a universal "faculty can read" rule

FERPA permits disclosure without consent to school officials whom the institution has determined to have legitimate educational interests. Employment or a faculty title alone is insufficient. The Department of Education describes legitimate educational interest generally as needing the record to fulfill professional responsibility.[5]

Implement role-based access plus relationship, purpose, lifecycle, and authority checks.

```
Access =
  authenticated actor
  + valid institutional membership
  + permitted capability
  + relevant course/caseload relationship
```

```
  + legitimate approved purpose
  + permitted record lifecycle
  + no active restriction
```

This expression is an engineering model. Institutional policy and counsel establish the legal basis.

4.2 Enforcement layers

Layer	What it enforces
Identity	Who is acting and whether the session is valid
Membership	Institution and current affiliation
Role/capability	What classes of operation are permitted
Relationship	Which course, student, case or office is in scope
Purpose	Why access is permitted under approved policy
Lifecycle	Draft, published, submitted, released, archived
Database RLS	Which rows the actor may access
Command validation	Allowed state transitions and field changes
Storage policy	Access to artifacts and derivative files
Search/projection policy	Prevent disclosure through indexes and summaries
Audit	Evidence of decisions, operations and required disclosures

4.3 Proposed role matrix

Record/action	Student	Faculty	TA	Registrar	Support
Published assignment	Own authorized course	Assigned course	Assigned course	Only separately authorized	No default content access
Assignment draft	No	Authorized author	Explicit authoring scope	No default	No
Private draft/notes	Owner	No default	No default	No default	No default
Submission	Own	Assigned course	Assigned grading scope	Separate authorized purpose	Explicit time-bound scope only
Draft grade	No	Authorized grader	Authorized grader	Separate scope	No default
Release feedback	Receive own	If release-authorized	Only explicit grant	Domain-specific authority	No
Enrollment modification	Request only	No generic authority	No	Approved command scope	No

Record/action	Student	Faculty	TA	Registrar	Support
Official grade posting	No	Submit where authorized	No generic authority	Approved institutional workflow	No
Bulk education-record export	Own permitted rights workflow	Explicit purpose and scope	Normally denied	Explicit scope	Normally denied

“Administrator” is not a universal content-access role. Infrastructure administration should be separated from education-record access.

4.4 Example policy specification

This is illustrative policy pseudocode—not an existing deployed policy.

```
ALLOW submission.read WHEN
  tenant_matches
  AND active_membership
  AND (
    actor_owns_submission
    OR current_teaching_assignment_has_submission_read
    OR current_grading_assignment_has_submission_read
  )

ALLOW grade.read WHEN
  tenant_matches
  AND (
    authorized_grader_for_course
    OR (
      actor_is_student_recipient
      AND grade_has_valid_release
    )
  )

ALLOW grade.release WHEN
  tenant_matches
  AND current_grant_has_grade_release
  AND course_scope_matches
  AND expected_version_matches
  AND required_reviews_complete
  AND no_active_release_hold

DENY private_document.read WHEN
  actor_is_not_owner
  AND no_current_explicit_share_grant
```

Purpose cannot be established by letting the client send `"legitimate educational interest": true`. It must derive from approved workflow and institutional relationships.

4.5 RLS and privileged-service controls

- Enable RLS on exposed academic tables.
- Test both positive and negative policies.
- Protect inserts and updates with appropriate checks.
- Restrict protected columns and state transitions.
- Keep privileged keys out of browser code.
- Ensure worker access is explicitly scoped and audited.
- Review functions and views for unintended privilege elevation.

Supabase secret/service-role credentials can bypass RLS; keeping them server-side is necessary but does not itself make every server operation safe. A privileged worker must enforce its own permitted scope or use a suitably restricted database role.[6] [7]

4.6 Consent, disclosure, and vendor authority

Keep separate records for:

- Legal disclosure basis.
- Institution-approved school-official access.
- Student-granted sharing scope.
- External recipient authorization.
- Required disclosure accounting.
- Technical access logs.

A student sharing setting is not a substitute for a valid FERPA disclosure basis. Conversely, consent is not required for every valid institutional school-official access. School-official outsourcing conditions include direct institutional control and restrictions on use and redisclosure.[8] [9]

Do not automatically give guardians access. Route eligibility and disclosure basis through the institution-approved policy.

4.7 Revocation sequence

1. Revoke membership, relationship, or grant in the authoritative policy store.
2. Block new protected reads and commands.
3. Cancel pending work that no longer has authority.
4. Re-evaluate queued jobs before execution.
5. Invalidate or revalidate caches and search access.
6. Expire artifact access according to approved controls.
7. Revoke delegated access.
8. Record completion and unresolved limitations.

Previously downloaded content cannot be technically “un-downloaded.” Revocation controls and contracts must state that limitation plainly.

5. Unified navigation and component system

5.1 Shared shell

```
Semester shell
├── Context bar
│   ├── Personal / institution context
│   ├── Authorized workspace
│   ├── Term
│   ├── Universal search
│   └── Connection/freshness status
├── Desktop side rail / mobile tab bar
├── Page header
├── Main workspace
├── Contextual actions/details
└── Account, privacy, devices and memberships
```

The project design system already specifies Ink chrome, Parchment content, a desktop side rail, labelled mobile tabs, a context bar, and shared navigation components. [10]

5.2 Canonical navigation

Student area	Destinations
Today	Today, Action Center, notifications
Plan	Calendar, course planning, registration
Learn	Courses, assignments, study, writing, sources, groupwork
Help	Advising, tutoring, support
Campus	Events, organizations, community, services, maps
Progress	Grades, degree, career, portfolio
You	Account, privacy, connections, devices, appearance

Faculty uses the same shell with Overview, Teaching, Assignments, Grading, Students, Messages, Resources, and Settings.

The supplied material contains seven desktop areas and five mobile destinations—Today, My Path, Search, Plan, Me. Treat these as presentation mappings over one destination registry, not competing route systems. [10]

5.3 Component composition

```
CourseWorkspace
├── ContextBar
├── SourceLine + ConnectionState
├── Tabs
├── AssignmentList
│   └── ObjectCard
├── NextSteps
└── ScreenGuide

FacultyGradingWorkspace
├── ContextBar
├── DataTable / grading queue
├── SubmissionViewer
├── RubricForm
├── SaveState
├── ActionPreview
├── ApprovalBanner
└── DecisionTrail
```

The proposed `SubmissionViewer` and `RubricForm` are domain components; the other named families appear in the supplied component inventory. [10]

5.4 Required component contract

Every domain component must declare:

- Accepted record types and versions.
- Required capabilities.
- Source and authority display.
- Loading, empty, error, forbidden, offline and stale behavior.
- Save and retry semantics.
- Keyboard and assistive-technology behavior.
- Consequence preview for protected actions.
- Audit-producing commands.
- Mobile layout.
- Privacy-sensitive fields.

Navigation visibility is never the authorization mechanism.

6. Legacy-record migration strategy

6.1 Migration order

```
Authorize
   ↓
Inventory and classify
   ↓
Map source → canonical schema
   ↓
Extract into restricted staging
   ↓
Resolve identities
   ↓
Transform idempotently
   ↓
Quarantine exceptions
   ↓
Reconcile and rehearse
   ↓
Shadow operation
   ↓
Approved domain cutover
   ↓
Observe, close out, retire temporary data
```

6.2 Domain ownership rules

Record type	Migration rule
Personal notes and plans	Student-controlled import; remain private
SIS enrollments	Preserve institutional authority and effective dates
LMS assignments	Preserve source, version, due-date interpretation and lifecycle
Submissions	Preserve student linkage, artifact checksum and acceptance meaning
Grades	Separate draft, released feedback and official grade status
Historical memberships	Do not activate access merely because a historical row exists
Prototype/sample records	Exclude from production
Company records	Keep outside institutional student-record scope

6.3 Identity resolution

- Prefer authoritative external identifiers.
- Require reviewed linking to current accounts.
- Never merge people solely by name.
- Do not treat an email-domain match as proof of course enrollment.
- Quarantine ambiguous matches.
- Preserve historical records without creating active privileges.

6.4 Reconciliation checks

A migration passes only when the approved checks cover:

- Counts by institution, term, offering and lifecycle.
- Correct student and faculty relationships.
- Tenant-consistent foreign keys.
- Source-to-canonical mapping uniqueness.
- Assignment versions and deadlines.
- Artifact availability and checksum.
- Draft versus released grade distinctions.
- Revoked and inactive identities.
- Access outcomes from each relevant role.
- Incremental change ordering and duplicate delivery.

Zero unresolved cross-tenant associations, unauthorized disclosure, or access-affecting identity ambiguity should be a proposed activation requirement. Other tolerances require institutional data-owner approval.

6.5 Cutover and rollback

Keep the legacy system authoritative until the institution explicitly approves a domain ownership change.

At cutover:

1. Record the source checkpoint.
2. Finish the baseline import.
3. Apply final incremental changes.
4. Reconcile and obtain acceptance.
5. Enable approved read/write behavior.
6. Verify representative student and faculty accounts.
7. Monitor stop criteria.

If rollback is required, stop affected actions and preserve new legitimate records. Reconcile writes made during the cutover interval rather than deleting imported records indiscriminately.

7. Required evidence before activation

Phase One is ready only when the release packet includes:

- Exact application, schema, worker and configuration versions.
- Successful backward-compatibility rehearsal.

- Tenant/course/role isolation tests.
- FERPA applicability and institutional access determinations.
- Student–faculty workflow acceptance.
- Migration reconciliation and exception acceptance.
- Accessibility and security review.
- Restore, rollback, revocation and offboarding exercises.
- Named support and incident responders.
- Approved tenant-specific activation decision.

The project's existing gate catalog supplies the broader completion categories; this runbook turns them into deployment and access-control obligations. Neither a successful build nor a rendered workflow substitutes for that evidence. [1]

⁂

1. The Main Semester design system (2) copy/docs/master/SEMESTER_RELEASE_GATE_CATALOG.md
2. https://supabase.com/docs/guides/database/connecting-to-postgres
3. https://vercel.com/docs/rolling-releases
4. https://vercel.com/docs/skew-protection
5. https://studentprivacy.ed.gov/faq/under-ferpa-may-educational-agency-or-institution-disclose-education-records-any-its-employees
6. https://supabase.com/docs/guides/database/postgres/row-level-security
7. https://supabase.com/docs/guides/database/secure-data
8. https://studentprivacy.ed.gov/faq/who-school-official-under-ferpa
9. https://studentprivacy.ed.gov/sites/default/files/resource_document/file/Vendor FAQ.pdf
10. The Main Semester design system (2) copy/readme.md
11. https://supabase.com/features/row-level-security
12. https://vercel.com/docs/deployment-checks
13. https://vercel.com/docs/deployments
14. https://vercel.com/changelog/skew-protection-max-age-now-supports-the-full-deployment-lifetime
15. https://tomodahinata.com/en/blog/vercel-deployments-cicd-rollback-rolling-releases-guide
16. https://studentprivacy.ed.gov/frequently-asked-questions
17. https://studentprivacy.ed.gov/ferpa